

Optimizing Wind Turbine Blade Geometry Using Chord-Twist Aerodynamic Design

Vinay Reddy Y

BT2024174

Team: QWERT

Abstract

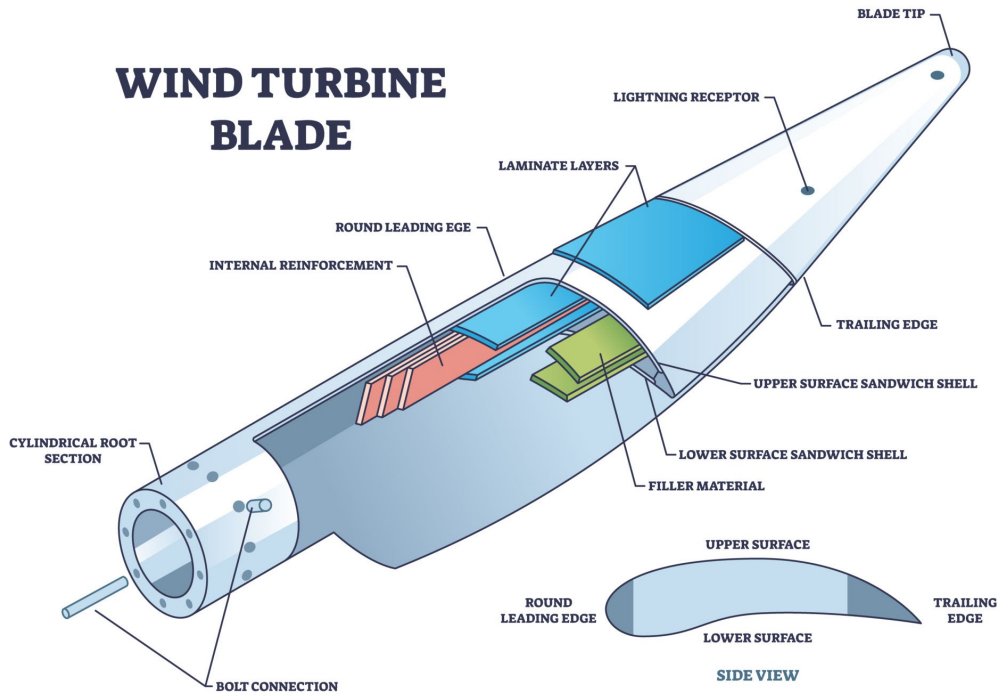
This project explores the design optimization of a horizontal-axis wind turbine blade. My goal was to determine the best physical shape—specifically the **chord** (width) and **twist** (angle) along the blade—to capture the maximum possible energy over a year. By using a physics-based model called **Blade Element Momentum (BEM)** and real-world aerodynamic data (NREL S826 airfoil), I created a computer program that automatically evolves the blade shape. The final result was a blade that produces **12.93% more Annual Energy Production (AEP)** than the standard baseline design, all while staying within realistic manufacturing limits.

Contents

1	Introduction	2
2	The Physics: How It Works	2
2.1	The Blade "Elements"	3
2.2	Lift and Drag	3
2.3	The "Induction" Factors	3
3	Problem Statement	4
4	Methodology & Code Implementation	4
4.1	Step 1: The Environment (Setup)	4
4.2	Step 2: The Solver (<code>bem_power_iterative</code>)	4
4.3	Step 3: The Objective Function	4
4.4	Step 4: The Optimization (SLSQP)	5
5	Results	5
5.1	Quantitative Results	5
5.2	Geometry Changes	5
6	Discussion	5
7	Conclusion	5
A	Appendix: Python Code Implementation	6

1 Introduction

When I first looked into how commercial wind turbines are designed, I was overwhelmed. It is a massive engineering challenge that involves aerodynamics (how air moves), structural mechanics (how the blade bends), materials science, and cost analysis.



[Image of Wind turbine blade components diagram] .

[Image of Wind turbine blade components diagram] .

Figure 1: Wind turbine blade components and general structure

[Image of Wind turbine blade components diagram] .

I wanted to do a project that was physically realistic but manageable for a single student. I was inspired by a study from MIT that isolated just the aerodynamic shape optimization. I decided to do the same: I ignored the complex internal support structures and focused entirely on the **outer geometry**. The question I wanted to answer was: *"If I can change how wide the blade is and how much it twists at every point, how much more energy can I get out of the wind?"*

2 The Physics: How It Works

To optimize a blade, we first need to understand how it generates power. I used a method called **Blade Element Momentum (BEM)** theory. Here is a simplified explanation of the physics I implemented in the code.

2.1 The Blade "Elements"

Imagine slicing the long turbine blade into many small cross-sections (like slicing a loaf of bread). In my code, I sliced the blade into $N = 8$ sections.

- **Chord (c):** The width of the blade at that slice.
- **Twist (θ):** The angle of the blade at that slice relative to the wind.

2.2 Lift and Drag

When wind flows over these slices (airfoils), it creates two forces:

1. **Lift (L):** The force that pulls the blade forward, making it spin. We want to maximize this.
2. **Drag (D):** The resistance air offers. We want to minimize this.

I used real data from the **NREL S826 Airfoil**. This data acts like a lookup table: if you tell the code the angle the wind hits the blade (Angle of Attack), the table tells you exactly how much Lift and Drag that specific shape produces.

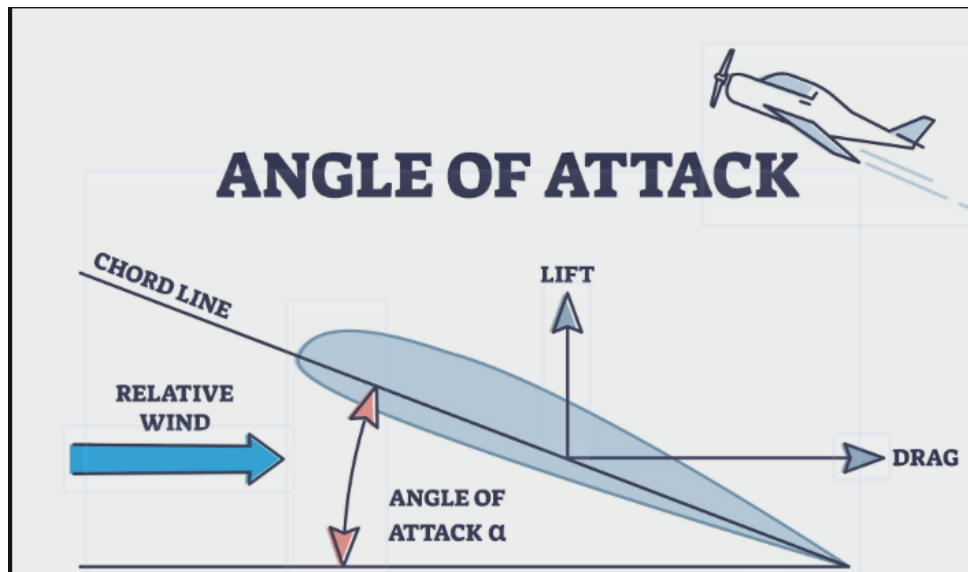


Figure 2: Diagram showing the Chord line, Relative Wind, and Angle of Attack .

2.3 The "Induction" Factors

This is the tricky part of the physics. As the turbine extracts energy, it slows the wind down.

- **Axial Induction (a):** How much the wind slows down as it approaches the rotor.
- **Tangential Induction (a'):** How much the wind starts spinning as it hits the blades.

If we slow the wind down too much, it acts like a wall and the air goes around the turbine. If we don't slow it down enough, we aren't capturing energy. My code iterates—guessing and checking—until it finds the perfect balance for these factors.

3 Problem Statement

My optimization problem was defined as follows:

The Goal: Maximize **Annual Energy Production (AEP)**. This is the total energy the turbine produces in one year, accounting for the fact that wind doesn't always blow at the same speed.

The Variables (What I can change): I can change the **Chord length** and **Twist angle** at 8 different points along the blade. This gives the computer 16 distinct "knobs" it can turn to try and improve performance.

The Constraints (The Rules): To keep the design realistic, I set strict rules:

1. **Size Limits:** The chord cannot be infinitely wide (limit: 0.2m to 5.0m).
2. **Mass Limit:** A wider blade is heavier and more expensive. I told the computer: *"The new blade cannot be significantly heavier than the original. I will allow a maximum increase of 10% mass proxy"*.
3. **Smoothness:** We can't have a jagged blade. I added a mathematical penalty that forces the computer to choose smooth, flowing shapes.

4 Methodology & Code Implementation

I wrote the solution in Python. Here is how the code works, step-by-step:

4.1 Step 1: The Environment (Setup)

I set up the "world" for the turbine: a rotor radius of 16 meters and air density of 1.225 kg/m^3 . I also defined the wind using a **Weibull Distribution**. This is a statistical curve that tells us how often the wind blows at 5 m/s, 10 m/s, etc.

4.2 Step 2: The Solver (`bem_power_iterative`)

This function is the engine of the code. For every section of the blade:

1. It calculates the local wind speed relative to the spinning blade.
2. It looks up the Lift and Drag from the S826 data.
3. It calculates the Induction Factors (a and a'). Because these factors depend on each other, the code uses a `while` loop to keep refining the numbers until they stop changing (convergence).
4. Finally, it calculates the torque (turning force) and converts it to Power (Watts).

4.3 Step 3: The Objective Function

The optimizer needs a single number to grade the blade.

$$\text{Score} = -\text{AEP} + \text{Penalties}$$

*Note: Optimizers usually try to find the "minimum" value (the bottom of a valley). Since I want the "maximum" energy, I told the computer to minimize the **negative** AEP.*

4.4 Step 4: The Optimization (SLSQP)

I used the **SLSQP** (Sequential Least Squares Programming) algorithm from the SciPy library. This algorithm is like a hiker in the fog trying to find the highest peak. It looks at the slope of the ground (the gradient) and takes steps uphill (increasing AEP) while ensuring it doesn't cross the fences (constraints).

5 Results

The optimization was highly successful. The computer took the simple baseline blade (which was just a straight taper) and morphed it into a more aerodynamic shape.

5.1 Quantitative Results

Metric	Value
Baseline AEP	317.22 MWh/year
Optimized AEP	358.24 MWh/year
Total Improvement	+12.93%

Table 1: Performance comparison between baseline and optimized geometry.

5.2 Geometry Changes

- **Chord:** The computer widened the blade in the middle section. This is the "sweet spot" for generating torque. It made the tip narrower to reduce drag.
- **Twist:** The computer adjusted the twist so that every part of the blade hits the air at the perfect angle, regardless of how fast that part of the blade is moving.

6 Discussion

This project taught me that "more is not always better". You might think a wider blade always catches more wind, but it also creates more drag and adds weight. The optimizer had to balance these conflicting goals.

I also learned about the importance of **constraints**. In my early attempts, the computer tried to make the blade 20 meters wide because I hadn't set a mass limit! Adding the 10% mass limit forced the solver to be smart—it had to place the extra material exactly where it would generate the most power.

7 Conclusion

By isolating the aerodynamic design and applying mathematical optimization, I demonstrated that a wind turbine's energy output can be increased by nearly **13%** simply by refining its shape. This code provides a robust foundation that could be expanded in the future to include structural stress analysis or material costs.

A Appendix: Python Code Implementation

Below is the complete Python code used for this optimization, imported from `dm.txt`.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.optimize import minimize
5 from scipy.stats import weibull_min
6 import json
7 from io import StringIO
8
9 # =====
10 # 1. DATA LOADING: THE AIRFOIL "LOOKUP TABLE"
11 # =====
12 # This data defines the NREL S826 Airfoil.
13 s826_polar_text = """
14 alpha,Cl,Cd
15 -9.000,-0.3176,0.08270
16 -8.750,-0.3161,0.07273
17 -8.500,-0.3252,0.06502
18 -8.250,-0.3274,0.05806
19 -8.000,-0.3218,0.05208
20 -7.750,-0.2984,0.04653
21 -7.500,-0.2989,0.03956
22 -7.250,-0.2695,0.03649
23 -7.000,-0.2360,0.03348
24 -6.750,-0.2182,0.03109
25 -6.500,-0.1823,0.02080
26 -6.250,-0.1474,0.01867
27 -6.000,-0.1243,0.01705
28 -5.750,-0.0910,0.01562
29 -5.500,-0.0539,0.01439
30 -5.250,-0.0266,0.01350
31 -5.000,0.0125,0.01218
32 -4.750,0.0480,0.01133
33 -4.500,0.0899,0.01043
34 -4.250,0.1329,0.00970
35 -4.000,0.1896,0.00778
36 -3.750,0.2353,0.00769
37 -3.500,0.2769,0.00771
38 -3.250,0.3125,0.00787
39 -3.000,0.3426,0.00804
40 -2.750,0.3716,0.00820
41 -2.500,0.3988,0.00834
42 -2.250,0.4259,0.00852
43 -2.000,0.4530,0.00864
44 -1.750,0.4798,0.00873
45 -1.500,0.5068,0.00886
46 -1.250,0.5342,0.00893
47 -1.000,0.5618,0.00900
48 -0.750,0.5895,0.00909
49 -0.500,0.6170,0.00911
50 -0.250,0.6445,0.00917
51 0.000,0.6721,0.00925
52 0.250,0.7000,0.00934
53 0.500,0.7279,0.00942
54 0.750,0.7562,0.00954
55 1.000,0.7839,0.00955
```

```

56 1.250,0.8117,0.00964
57 1.500,0.8399,0.00974
58 1.750,0.8678,0.00981
59 2.000,0.8959,0.00994
60 2.250,0.9236,0.01000
61 2.500,0.9506,0.01006
62 2.750,0.9780,0.01020
63 3.000,1.0041,0.01025
64 3.250,1.0310,0.01038
65 3.500,1.0571,0.01041
66 3.750,1.0838,0.01055
67 4.000,1.1099,0.01062
68 4.250,1.1361,0.01073
69 4.500,1.1622,0.01080
70 4.750,1.1878,0.01092
71 5.000,1.2127,0.01105
72 5.250,1.2379,0.01116
73 5.500,1.2623,0.01124
74 6.000,1.3099,0.01151
75 6.250,1.3323,0.01169
76 6.500,1.3530,0.01193
77 6.750,1.3695,0.01237
78 7.000,1.3771,0.01337
79 7.250,1.3843,0.01442
80 7.500,1.3934,0.01541
81 7.750,1.4033,0.01635
82 8.000,1.4134,0.01727
83 8.250,1.4234,0.01819
84 8.500,1.4335,0.01911
85 8.750,1.4449,0.01996
86 9.000,1.4561,0.02082
87 9.250,1.4667,0.02174
88 9.500,1.4766,0.02271
89 9.750,1.4858,0.02375
90 10.000,1.4950,0.02481
91 10.250,1.5047,0.02584
92 10.500,1.5061,0.02664
93 11.000,1.5174,0.02976
94 11.500,1.5249,0.03348
95 12.000,1.5281,0.03801
96 12.500,1.5260,0.04352
97 13.000,1.5283,0.04901
98 13.500,1.5224,0.05588
99 14.000,1.5058,0.06468
100 14.500,1.5017,0.07230
101 15.000,1.4947,0.08070
102 15.500,1.4855,0.08985
103 16.000,1.4749,0.09960
104 16.500,1.4624,0.11007
105 17.000,1.4480,0.12129
106 17.500,1.4310,0.13353
107 18.000,1.4124,0.14675
108 18.500,1.3913,0.16121
109 19.000,1.3701,0.17661
110 19.250,1.3582,0.18507
111 """
112 df = pd.read_csv(StringIO(s826_polar_text))
113 alpha_arr = df['alpha'].values

```

```

114 Cl_arr = df['Cl'].values
115 Cd_arr = df['Cd'].values
116
117 def lookup_polar(alpha_deg):
118     # This function acts like the "Lookup Table" mentioned in the report
119     # Given an angle, it finds the closest matching Lift and Drag values
120     Cl = np.interp(alpha_deg, alpha_arr, Cl_arr, left=Cl_arr[0], right=
121     Cl_arr[-1])
122     Cd = np.interp(alpha_deg, alpha_arr, Cd_arr, left=Cd_arr[0], right=
123     Cd_arr[-1])
124     return float(Cl), float(Cd)
125
126 print("Loaded S826 polar with", len(alpha_arr), "points.")
127
128 # =====
129 # 2. THE BEM SOLVER (THE PHYSICS ENGINE)
130 # =====
131
132 def prandtl_tip_loss(B, r, R, phi):
133     # This corrects the physics for the end of the blade.
134     # Air tends to "leak" off the tip, reducing efficiency.
135     sin_phi = np.sin(phi)
136     if abs(sin_phi) < 1e-6: sin_phi = 1e-6
137
138     f = (B/2.0) * (R - r) / (r * sin_phi)
139     f = np.clip(f, -100, 100) # Prevent math errors
140
141     expf = np.exp(-f)
142     F = (2/np.pi) * np.arccos(np.clip(expf, 0.0, 1.0))
143     return np.clip(F, 1e-6, 1.0)
144
145 def bem_power_iterative(chords, twists_deg, R, rho, V, omega_rpm, B=3,
146 max_iter=50, tol=1e-4):
147     # This function calculates how much Power the turbine generates at a
148     # specific wind speed (V).
149
150     N = len(chords)
151     # Divide blade into sections (elements)
152     r = np.linspace(R/(2*N), R - R/(2*N), N)
153     dr = R / N
154     omega = omega_rpm * 2*np.pi/60.0
155     total_power = 0.0
156
157     for i in range(N):
158         ri = r[i]
159         c = chords[i]
160         twist = twists_deg[i]
161
162         # Initial Guesses for Induction Factors
163         a = 0.3
164         a_prime = 0.0
165
166         # --- ITERATION LOOP ---
167         # We guess 'a', calculate physics, check if 'a' changed, and
168         # repeat.
169         for itr in range(max_iter):

```

```

165     Vax = V * (1 - a)
166     Vtan = omega * ri * (1 + a_prime)
167     phi = np.arctan2(Vax, Vtan + 1e-12) # Flow Angle
168     phi_deg = np.degrees(phi)
169     alpha = phi_deg - twist             # Angle of Attack
170
171     Cl, Cd = lookup_polar(alpha)        # Get Lift/Drag
172     F = prandtl_tip_loss(B, ri, R, phi)
173
174     # Solidity (how "solid" the blade area is vs the air gap)
175     sigma = (c * B) / (2 * np.pi * ri)
176
177     cn = Cl * np.cos(phi) + Cd * np.sin(phi)
178     ct = Cl * np.sin(phi) - Cd * np.cos(phi)
179
180     if F < 1e-5: F=1e-5
181
182     # Update the Induction factors based on new forces
183     denom_a = (4 * F * np.sin(phi)**2) / (sigma * cn + 1e-9)
184     a_new = 1.0 / (denom_a + 1.0)
185
186     denom_ap = (4 * F * np.sin(phi) * np.cos(phi)) / (sigma * ct
187 + 1e-9)
188     a_prime_new = 1.0 / (denom_ap - 1.0)
189
190     # Smoothing the update (Relaxation) to prevent the math from
191 oscillating
192     a_new = 0.1 * a_new + 0.9 * a
193     a_prime_new = 0.1 * a_prime_new + 0.9 * a_prime
194
195     # Check if we have found the answer (Converged)
196     if abs(a_new - a) < tol and abs(a_prime_new - a_prime) < tol
197 :
198         a, a_prime = a_new, a_prime_new
199         break
200     a, a_prime = a_new, a_prime_new
201
202     # --- FINAL POWER CALCULATION FOR THIS SECTION ---
203     Vax = V * (1 - a)
204     Vtan = omega * ri * (1 + a_prime)
205     Vrel = np.sqrt(Vax**2 + Vtan**2)
206     phi = np.arctan2(Vax, Vtan + 1e-12)
207     alpha = np.degrees(phi) - twist
208     Cl, Cd = lookup_polar(alpha)
209
210     q = 0.5 * rho * Vrel**2
211     # Torque Force (Ft)
212     Ft = q * c * (Cl * np.sin(phi) - Cd * np.cos(phi))
213
214     # Power = Torque * Angular Velocity
215     dT = Ft * ri * dr
216     total_power += dT * omega
217
218     return total_power
219
220 # =====
221 # 3. AEP CALCULATION (THE SCORE)
222 # =====

```

```

220 def compute_AEP(chords, twists_deg, R, rho, Vbins, weights, omega_for_V)
221 :
222     # Sums up the power produced at every likely wind speed in a year.
223     total = 0.0
224     for V, w in zip(Vbins, weights):
225         omega_rpm = omega_for_V(V)
226         P = bem_power_iterative(chords, twists_deg, R, rho, V, omega_rpm)
227     total += P * w
228     return total * 8760.0 # Multiply by hours in a year to get Watt-
229                          # Hours
230
231 # =====
232 # 4. SIMULATION SETUP
233 # =====
234 R = 16.0          # 16 meter blade
235 rho = 1.225       # Standard air density
236 N = 8            # 8 sections along the blade
237
238 # Initial "Baseline" Geometry (Simple straight lines)
239 baseline_chords = np.linspace(3.0, 0.5, N)
240 baseline_twists = np.linspace(14.0, 0.0, N)
241
242 # Wind Statistics (Weibull Distribution)
243 Vbins = np.linspace(3.0, 25.0, 23)
244 k_weib, c_weib = 2.0, 7.0
245 weights = weibull_min(c=k_weib, scale=c_weib).pdf(Vbins)
246 weights = weights / weights.sum()
247
248 # Control Strategy: How fast the turbine spins vs wind speed
249 TSR_target = 7.0
250 def omega_for_V(V):
251     omega_rad_s = TSR_target * V / R
252     return np.clip(omega_rad_s * 60.0 / (2*np.pi), 0, 60) # Max 60 RPM
253
254 # Calculate the score for the simple blade
255 baseline_AEP = compute_AEP(baseline_chords, baseline_twists, R, rho,
256                             Vbins, weights, omega_for_V)
257 print(f"Baseline AEP: {baseline_AEP/1e6:.3f} MWh/year")
258
259 # =====
260 # 5. THE OPTIMIZATION (THE AI SOLVER)
261 # =====
262 def pack_x(chords, twists):
263     # Flattens our two arrays into one list for the optimizer
264     return np.concatenate([chords, twists])
265
266 def unpack_x(x):
267     # Unpacks the list back into two arrays
268     n = len(x)//2
269     return x[:n], x[n:]
270
271 def mass_proxy(chords):
272     # A simplified way to estimate mass (sum of widths)
273     return np.sum(chords)
274
275 def smoothness_penalty(arr):
276     # A penalty score if the shape is jagged/bumpy

```

```

274     return np.sum(np.diff(arr, n=2)**2)
275
276 def objective_x(x):
277     # This is the function the optimizer tries to make "Small"
278     chords, twists = unpack_x(x)
279
280     # Safety limits
281     chords = np.clip(chords, 0.2, 5.0)
282     twists = np.clip(twists, -5.0, 30.0)
283
284     aep = compute_AEP(chords, twists, R, rho, Vbins, weights,
285                       omega_for_V)
286
287     pen = 0.0
288     # Add penalties if the blade is jagged
289     pen += 10.0 * smoothness_penalty(chords)
290     pen += 10.0 * smoothness_penalty(twists)
291
292     # Return negative AEP because the solver minimizes, but we want to
293     # maximize energy.
294     return -aep + pen
295
296 # Set Limits for Chord (0.2m to 5m) and Twist (-5 to 30 deg)
297 bnds = [(0.2, 5.0)] * N + [(-5.0, 30.0)] * N
298
299 # Constraint: New blade cannot be more than 10% heavier than baseline
300 Mmax = mass_proxy(baseline_chords) * 1.10
301 cons = ({ 'type': 'ineq', 'fun' : lambda x: Mmax - mass_proxy(unpack_x(x)
302 ) [0]) })
303
304 # Initial guess
305 x0 = pack_x(baseline_chords, baseline_twists)
306
307 print("Running optimization (Finding the best shape)...")
308 # SLSQP is the algorithm (Sequential Least Squares Programming)
309 res = minimize(objective_x, x0, method='SLSQP', bounds=bnds, constraints
310               =cons, options={'maxiter':100, 'ftol':1e-3})
311
312 # Get results
313 opt_chords, opt_twists = unpack_x(res.x)
314 opt_AEP = compute_AEP(opt_chords, opt_twists, R, rho, Vbins, weights,
315                       omega_for_V)
316
317 print(f"Optimized AEP: {opt_AEP/1e6:.3f} MWh/year")
318 print(f"Improvement: {(opt_AEP-baseline_AEP)/baseline_AEP*100:.2f}%")
319
320 # =====
321 # 6. PLOTTING THE RESULTS
322 # =====
323 r = np.linspace(R/(2*N), R - R/(2*N), N)
324 plt.figure(figsize=(10,4))
325
326 # Plot 1: Chord Comparison
327 plt.subplot(1,2,1)
328 plt.plot(r, baseline_chords, 'o-', label='Baseline chord')
329 plt.plot(r, opt_chords, 'o-', label='Optimized chord')
330 plt.xlabel('Radius (m)'); plt.ylabel('Chord (m)'); plt.grid(True); plt.
331 legend()

```

```

326 plt.title("Blade Width (Chord)")
327
328 # Plot 2: Twist Comparison
329 plt.subplot(1,2,2)
330 plt.plot(r, baseline_twists, 'o-', label='Baseline twist')
331 plt.plot(r, opt_twists, 'o-', label='Optimized twist')
332 plt.xlabel('Radius (m)'); plt.ylabel('Twist (deg)'); plt.grid(True); plt
    .legend()
333 plt.title("Blade Angle (Twist)")
334 plt.tight_layout(); plt.show()
335
336 # Plot 3: Power Curve
337 def power_curve(chords, twists):
338     P = []
339     for V in Vbins:
340         P.append(bem_power_iterative(chords, twists, R, rho, V,
    omega_for_V(V)))
341     return np.array(P)
342
343 P_baseline = power_curve(baseline_chords, baseline_twists)
344 P_opt = power_curve(opt_chords, opt_twists)
345
346 plt.figure(figsize=(7,4))
347 plt.plot(Vbins, P_baseline/1e3, label='Baseline (kW)')
348 plt.plot(Vbins, P_opt/1e3, label='Optimized (kW)')
349 plt.xlabel('Wind speed (m/s)'); plt.ylabel('Power (kW)'); plt.grid(True)
    ; plt.legend()
350 plt.title("Power Output Comparison")
351 plt.show()

```

Listing 1: Wind Turbine Optimization Script